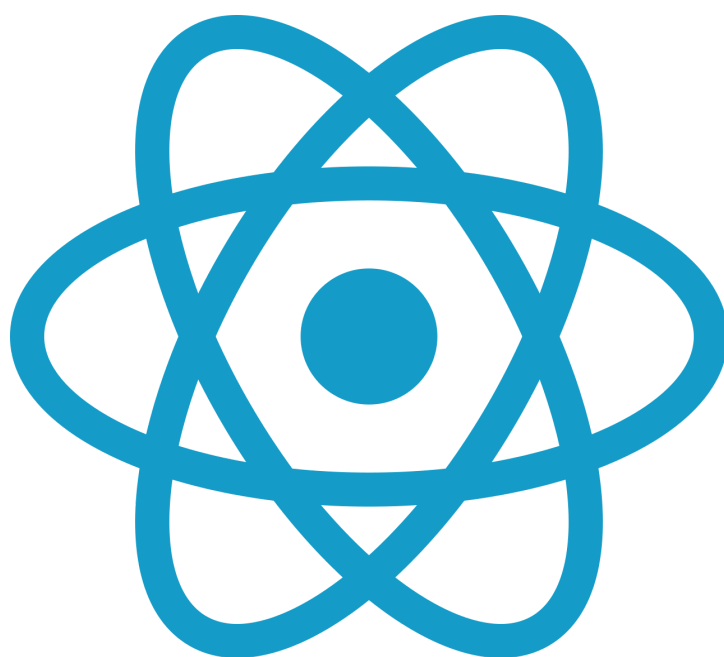


atlántida

Formación Profesional

REACT



ANTONIO CASADO VICO
OLIVER MATTHEW RUTTER FRANCO
RAFAEL OSUNA RAMÍREZ
CRISTINA RAMOS MORA
SERGIO HERNÁNDEZ VAQUERO
ANTONIO JOSE HEREDIA FERNÁNDEZ

A → ¿Qué es React? ¿Para qué sirve? ¿Quién lo mantiene?.....	3
B → Conocimientos y herramientas.....	3
C → Ventajas de usar React:.....	3
D → Inconvenientes de usar React:.....	4
E → Uso recomendado:.....	4
F → Comparación:.....	4
Organización de la interfaz.....	5
Acciones que resolvería el navegador (cliente).....	6
Datos y validaciones del servidor.....	7
- Inicio de sesión.....	7
- Datos del perfil y expediente.....	7
- Formulario de inscripción.....	7
- Cursos, asignaturas y filtros.....	8
Web-Grafía.....	9

A → ¿Qué es React? ¿Para qué sirve? ¿Quién lo mantiene?

Es una librería JavaScript de código abierto enfocada a la visualización. Esta tecnología nos permite desarrollar interfaces de usuario fácilmente, esto es gracias a componentes interactivos y reutilizables.

React sirve para construir interfaces de usuario a partir de piezas independientes, los componentes. Puedes crear tus propios componentes de React como Thumbnail, LikeButton y Video. Luego se combina para formar pantalla, páginas y aplicaciones.

Hasta 2026 fue mantenido por Meta. Desde febrero de 2026 pasó a ser gestionado por la React Foundation, una fundación independiente alojada por la Linux Foundation, con Meta, Amazon, Microsoft, Vercel y otras empresas como miembros fundadores.

B → Conocimientos y herramientas

- Conocimientos previos:
Es fundamental contar con conocimientos de HTML, CSS, Javascript y sería recomendable Git.
- Lenguajes utilizados:
Los lenguajes principales obligatorios serían Javascript y JSX y sería recomendable conocer Typescript.
- Herramientas:
Next.js que es un framework, React Developer Tools que sirve para inspeccionar el árbol de componentes. Node.js para poder instalar dependencias y usar compiladores en el equipo local.

C → Ventajas de usar React:

- Es posible dividir la interfaz en bloques pequeños e independientes para reutilizarlos en varias partes del proyecto, ahorrando tiempo y facilitando el mantenimiento.
- Hay un flujo de datos unidireccional, que viaja hacia en una sola dirección. Esto hace que el código sea más predecible y más fácil de corregir los fallos.
- React es una DOM virtual. Quiere decir que React es una copia virtual de DOM. Solo se actualizan las partes necesarias en la página real los cambios realizados en la copia anteriormente, mejorando así su rendimiento.

D → Inconvenientes de usar React:

- Existe una falta de estructura oficial al no imponer una organización efectiva, en el que cada equipo decide cómo estructurar el código, lo que deriva a proyectos desordenados y poco mantenimiento eficiente.
- Tiene una curva de aprendizaje complicada, ya que mezclar HTML con la lógica de JS mediante JSX, eso requiere un dominio sólido del lenguaje.
- Un ecosistema en constante cambio. Significa que las herramientas oficiales evolucionan rápido, exigiendo una actualización constante.

En resumen, React es una librería minimalista y no un framework completo, lo que obliga a depender de herramientas externas.

E → Uso recomendado:

Elegiríamos React para una aplicación que necesite mucha interactividad y estado complejo con el cliente, como una plataforma de gestión con dashboards, filtros dinámicos y actualizaciones a tiempo real (Ej: un panel de administración o red social). Aquí se encuentra la flexibilidad de React para elegir tus propias herramientas, es decir, con React hay que elegir e instalar por separado piezas como el router o el gestor de estado (esfuerzo extra), pero a cambio el proyecto queda hecho a medida de lo que necesita, en vez de ajustarse a lo que un framework cerrado ya trae impuesto.

No la elegiríamos para una web sencilla que solo da información, sin apenas interactividad (ej: la página de un negocio local). Montar todo lo que necesita React para eso sería complicarlo sin motivo, y encima la web cargaría más lenta sin necesitarlo; para eso valdría con HTML/CSS o una plantilla de página estática.

F → Comparación:

- Su arquitectura: React es una librería completa enfocada en la interfaz de usuario mientras que Angular es un framework completo que trae sistemas de rutas, cliente HTTP, gestión de formularios y CLI completo.
- Lenguaje y sintaxis: React utiliza JSX que combina Javascript con una estructura similar a HTML en el mismo archivo, mientras que Angular exige un uso estricto de Typescript que separa la lógica de su plantilla HTML.

Organización de la interfaz

La aplicación se dividiría en componentes independientes y reutilizables:

- **Login:** mostraría el formulario de inicio de sesión con los campos de usuario y contraseña. También incluiría un enlace o botón para acceder al formulario de inscripción.
- **Formulario:** permitiría introducir los datos necesarios para realizar la inscripción. Este componente controlaría los campos y mostraría posibles errores de formato.
- **Home:** actuaría como contenedor de la página principal después de iniciar sesión.
- **HomeHero:** mostraría la bienvenida y el área principal del alumnado.
- **SeleccionCursos:** recibiría y mostraría la lista de cursos o asignaturas disponibles para el alumno.
- **FiltrosBusqueda:** incluiría la barra de búsqueda y los filtros de cursos. Los valores introducidos se comunicarían al componente que contiene la lista de asignaturas.
- **PerfilAlumno:** mostraría la información del alumno cuando se seleccionase el usuario.
- **Footer:** contendría la información común de la aplicación, como enlaces, avisos o datos de contacto.

El componente **Home** podría mantener parte del estado común, como el usuario autenticado, los filtros seleccionados y la lista de asignaturas. Después, pasaría esos datos y las funciones necesarias a FiltrosBusqueda, SeleccionCursos y PerfilAlumno mediante propiedades. React recomienda colocar el estado compartido en el componente padre común y pasar los datos y controladores a los componentes hijos.

Acciones que resolvería el navegador (cliente)

El navegador se encargaría principalmente de mostrar la interfaz y responder a las interacciones del usuario:

- Mostrar inicialmente la pantalla de Login.
- Detectar cuándo el usuario escribe en los campos del login o del formulario.
- Comprobar aspectos básicos del formulario, como campos obligatorios, formato del correo electrónico o longitud mínima de la contraseña.
- Enviar las credenciales al servidor cuando se pulse el botón de iniciar sesión.
- Mostrar mensajes de error si faltan datos o si el servidor devuelve un error.
- Cambiar de Login a Home cuando el servidor confirme que las credenciales son correctas.
- Mostrar el formulario de inscripción cuando el usuario pulse el enlace correspondiente.
- Actualizar el texto de búsqueda y los filtros en tiempo real.
- Solicitar o filtrar la lista de asignaturas según los criterios seleccionados.
- Mostrar el perfil del alumno al pulsar sobre el usuario.
- Representar visualmente estados de carga, resultados vacíos y errores.

Por ejemplo, al escribir Bases de datos en la barra de búsqueda, FiltrosBusqueda actualizaría su estado y comunicaría el valor a SeleccionCursos. La lista podría actualizarse inmediatamente si los cursos ya están cargados, o solicitar nuevos resultados al servidor si la búsqueda se realiza mediante una API.

Datos y validaciones del servidor

El servidor sería responsable de gestionar la información sensible y comprobar que las operaciones están autorizadas.

- Inicio de sesión

Cuando el navegador envíe el usuario y la contraseña, el servidor debería:

- Comprobar que los datos tienen el formato esperado.
- Buscar la cuenta correspondiente.
- Comparar la contraseña con su versión almacenada de forma segura.
- Crear una sesión o devolver un mecanismo de autenticación seguro.
- Rechazar las credenciales incorrectas sin revelar información innecesaria.
- Devolver al navegador únicamente la información necesaria para continuar.

El navegador no debería decidir por sí mismo que el login es correcto. Solo el servidor puede confirmar que las credenciales son válidas.

- Datos del perfil y expediente

El servidor también tendría que comprobar la identidad del usuario en cada petición relacionada con sus datos. Por ejemplo, al solicitar el expediente o las asignaturas, debería:

- Verificar que existe una sesión válida.
- Identificar al alumno autenticado.
- Comprobar que ese alumno tiene permiso para consultar la información solicitada.
- Devolver únicamente sus propios datos.
- Evitar que un usuario pueda cambiar un identificador en la URL para consultar el expediente de otra persona.

Las comprobaciones de autorización deben realizarse en el servidor y no solo ocultando componentes en React, porque los controles del cliente pueden modificarse o saltarse.

- Formulario de inscripción

Cuando se envíe el formulario, el servidor tendría que validar de nuevo todos los datos:

- Nombre y apellidos.
- Correo electrónico.
- Curso seleccionado.
- Datos académicos necesarios.
- Campos obligatorios.
- Longitud y formato de cada valor.
- Posibles duplicados o inscripciones incompatibles.

También debería comprobar que el curso existe y que el alumno tiene permiso para inscribirse. La validación del servidor es necesaria porque cualquier validación realizada exclusivamente con JavaScript puede ser evitada por el usuario.

- Cursos, asignaturas y filtros

El servidor podría encargarse de:

- Consultar los cursos y asignaturas en la base de datos.
- Comprobar el expediente del alumno.
- Devolver sólo las asignaturas que le corresponden.
- Aplicar los filtros recibidos.
- Validar que los parámetros de búsqueda tienen un formato correcto.
- Limitar el número de resultados devueltos.
- Evitar consultas manipuladas o accesos a información no autorizada.

Razones y riesgos de React

Hemos elegido React frente a Vue y Angular porque ofrece una mayor flexibilidad para organizar la aplicación. React **está centrado en la creación de interfaces mediante componentes reutilizables, pero no obliga a utilizar una arquitectura tan cerrada como Angular**. Esto nos permite diseñar una estructura adaptada a nuestro proyecto, formada por componentes como Login, Formulario, HomeHero, SeleccionCursos, FiltrosBusqueda, PerfilAlumno y Footer.

Otro motivo es que React **permite reutilizar conocimientos de JavaScript y aprender la tecnología de forma progresiva**. Sus conceptos principales, como componentes, propiedades, estado, eventos y renderizado condicional, encajan directamente con las funciones de nuestro sistema: validar el login, mostrar la página principal, consultar el perfil del alumno y actualizar las asignaturas según los filtros. Frente a Angular, que incluye más herramientas y una estructura más completa desde el principio, React nos **permite comenzar con una solución más sencilla y añadir funcionalidades conforme el proyecto crezca**.

Un riesgo que habría que gestionar es **la seguridad de los datos**. No se debe confiar únicamente en las validaciones realizadas en React, **porque estas se ejecutan en el navegador y pueden modificarse**. Las credenciales, los datos del formulario y el expediente del alumno deben validarse también en el servidor. Además, las contraseñas deben almacenarse de forma segura, el acceso al expediente debe estar autorizado y las dependencias del proyecto deben mantenerse actualizadas para reducir posibles vulnerabilidades.

Web-Grafía

<https://es.react.dev/>

<https://react.dev/blog/2026/02/24/the-react-foundation>

<https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-react-foundation>

<https://ebac.mx/blog/que-es-react>

<https://www.qindel.com/conoces-react-y-sus-beneficios-si-no-es-asi-te-lo-explicamos/>

<https://www.soyhenry.com/blog/entendiendo-react-fundamentos-y-ventajas-de-este-popular-framework-front-end>

<https://openwebinars.net/blog/react-desde-cero-todo-lo-que-debes-saber/>

https://developer.mozilla.org/es/docs/Learn_web_development/Core/Frameworks_libraries/React_getting_started

<https://www.pontia.tech/que-es-react/>